

the script will move on to the next function, which is *backupDocRoot*.

4.

```
function backupDocRoot() {
    'which tar' -c -f "$BACKUP_DIR"/doc_root_backup-$(date
    +%d-%m-%y).tar.bz2 --directory="$DOC_ROOT" .
    if [[ -e "$BACKUP_DIR"/doc_root_backup-$(date +%d-%m-
    %y).tar.bz2 ]]; then
        backupDatabase
    else
        echo "Backup
        failed for document root on your server. Please check
        manually" | mail -s "Backup failed - Document Root" user@
        domain.com
    fi
}
```

This function will initiate a backup of the document root of your Web server (defined in one of the variables above) and create a compressed bz2 archive. Then it will check if a bz2 archive exists in the backup directory and only if it does, will it go forward onto the next function, which is *backupDatabase*. If, for some reason, the archive is not there, or it failed, the script will halt and an email will be dispatched to *user@domain.com*. You can certainly edit the contents of the email to be sent.

5.

```
function backupDatabase() {
    'which mysqldump' -u "$SDB_USER" -p"$SDB_PASSWORD"
    "$SDB_NAME" > "$BACKUP_DIR"/"$SDB_NAME"-"$(date +%d-%m-%y).sql" |
    tee "$BACKUP_DIR"/backup.log
    tar -c -f "$BACKUP_DIR"/"$SDB_NAME"-"$(date +%d-%m-
    %y).tar.bz2 "$BACKUP_DIR"/"$SDB_NAME"-"$(date +%d-%m-%y).sql"
    --remove-files 2> $BACKUP_DIR/tar.log
    if [[ -e "$BACKUP_DIR"/"$SDB_NAME"-"$(date +%d-%m-%y).
    tar.bz2 ]]; then
        createTar
    else
        echo "Backup
        failed for database on your server. Please check manually" |
        mail -s "Backup failed - Database" user@domain.com
    fi
}
```

This function block does two things. First, it takes a backup of the MySQL database in the *.sql* format, places it inside *BACKUP_DIR* and creates an archive with bz2 compression. After the archive is created, it will delete the original *.sql* file. If the bz2 compression is completed, the script will jump to the *createTar* function and if not, an email will be dispatched to the user.

6.

```
function createTar() {
    cd "$BACKUP_DIR"
    'which tar' -c -f backup-$(date +%d-%m-%y).tar doc_root_
    backup-$(date +%d-%m-%y).tar.bz2 "$SDB_NAME"-"$(date +%d-%m-%y).
    tar.bz2 --remove-files 2> final.log
    deleteOldBackup
}
```

This function will simply put two backups (database and document root) in a single *.tar* file. The archive which is created by this function is not compressed, but merely puts two backups under one file.

7.

```
function deleteOldBackup() {
    'which find' "$BACKUP_DIR"/"$SDB_NAME"* -mtime +"$DAYS_
    TO_KEEP_BACKUP" -exec rm {} \; >> "$BACKUP_DIR"/delete.log
}
```

As the name suggests, this will search for any backup file (*.tar*) which is *\$DAYS_TO_KEEP_BACKUP* days older and delete it. For example, if the value of *\$DAYS_TO_KEEP_BACKUP* is set to '5', then any backup file which is older than five days will be deleted.

Future possible amendments

This section explains how you can amend this script to get the most out of it and edit it according to your preferences. As you can see, the basic nature of this script is modular, and you may add other functions to it and call them to perform more complex tasks.

For example, if the size of your combined backup (database and document root) is comparatively low, you might want to email it instead of saving it on the server itself.

Another example could be to upload the backup to your Dropbox account. For this purpose, you would need to generate API keys. So Google these!

Yet another example would be to store the backup on an NFS share so that it is placed on another server.

Wrapping up

Numerous backup solutions are available, free or otherwise; so choose one that best meets your requirements, keeping in mind the server overhead that commercial backup tools tend to create. This is where shell scripts come in handy. They achieve the primary backup goal with minimal overhead. As they say, "There are multiple ways to do a task in Linux." 

By: Aman Hanjrah

The author is a postgraduate in networking and protocol design. He is currently working as a Linux systems administrator at Kayako Helpdesk. He blogs at <http://technlinux.net/> and can be reached at aman.hanjrah@technlinux.net.